GROUP 3: CAMPUSBOOK

Week 3 Group Project Submission

GitHub repository	https://github.com/Gerber-W/SPM_Group_Project
Project Supervisor	Jabulile Mashibini
Client	CTU Training Solutions
Submitted	Monday 19 October 2026

Name	Role
Willem Gerber	Group Lead
Oscar Khumalo	Member
Thomas du Plessis	Member
Lebohang Tlaitlai	Member
Gerhardus van den Berg	Member

Chapter 1: Introduction

Background of the Study

Consultations give students time with a lecturer outside class to ask questions get feedback and work through problems Research shows this support is often under used and that the reasons are usually practical rather than academic (Griffin, 2014)

At CTU Training Solutions arranging a consultation is informal Students send an email a Teams or WhatsApp message or simply knock on the lecturer door during posted hours Each lecturer handles this differently and nobody has a clear view of who is available who is coming or which meetings took place

Other sectors have solved a similar problem with online booking in healthcare web based appointment systems have reduced no-shows and waiting times and saved staff time (Zhao, 2017) and automated reminders improve attendance even further (Guy, 2012) With 96% of South African households owning a mobile phone (Statistics South Africa, 2024) a booking system that works well on a phone can reach almost every student

Problem Statement

CTU Training Solutions has no single reliable way for students to book consultations with lecturers Requests are spread across email Teams WhatsApp and walk-ins so students wait outside offices messages go unanswered and meetings are missed or double-booked Lecturers cannot plan their time because they do not know who is coming or why the problem peaks before tests and assignment deadlines exactly when students need help most

It affects students who need academic support and lecturers who juggle requests from several channels Our group has experienced this first hand our lecturer reviewed the proposal and agreed it addresses a real need and a short student survey will show how widespread it is

Research Aim

To design develop and evaluate CampusBook a web based system that lets CTU students book change and cancel consultations and lets lecturers manage their availability so that fewer consultations are missed or left unplanned

Research Objectives

1. Analyse how consultations are arranged today through a survey of at least 10 students by 21 October 2026
2. Compare at least three existing scheduling systems on their features strengths and limitations by 19 October 2026
3. Design and develop a web system with all six must-have features working on a live database by 23 October 2026
4. Test the system so that every must-have test case passes a booking takes under 2 minutes and pages load within 3 seconds
5. Evaluate the system with at least 5 students and 2 lecturers with 80% or more rating it easy to use

Research Questions

1. How do students and lecturers at CTU arrange consultations now and what problems do they face?
2. What do existing scheduling systems offer and what do they lack for a campus setting?
3. How can a web system let students book consultations and lecturers manage their time?
4. Does the system meet its functional and non-functional requirements?
5. Do students and lecturers find the system easy to use and useful?

Proposed Software Solution

CampusBook is a responsive web application with two roles Lecturers publish their weekly consultation slots and see who has booked and why Students browse open slots book one with a short reason for the visit and get an email reminder before the meeting The six must have features are:

1. Login with separate student and lecturer roles
2. Lecturers publish weekly consultation slots
3. Students book change or cancel a slot with a reason for the visit
4. Email confirmation and a reminder before each booking
5. Booking history for students and lecturers
6. Mobile friendly design

Scope

In scope	Out of scope
One CTU campus	Other campuses or institutions
Student and lecturer roles	Admin reporting dashboards
Six must have features with email reminders	Video calls SMS and native mobile apps
Test data created by the team	Links to CTU live student systems

Significance of the Study

- Students can see when lecturers are free get confirmed bookings and stop wasting trips before assessments
- Lecturers control their consultation time and know in advance who is coming and why
- CTU Training Solutions gains a low-cost tool that supports student success and evidence on whether online booking works as well on campus as it does in healthcare

Chapter 2: Literature Review

Existing Research

Consultations are under used Griffin (2014) surveyed undergraduate students at a large American university and found that most of the factors that stop students attending office hours are outside the lecturer control The one exception was feedback: students who found it useful came back more often A summary of similar survey work found that 66% of more than 600 students had never attended office hours for a course and only 8% went more than once a month (Weimer, 2015) Both sources show the problem clearly but they focus on student attitudes rather than on how meetings are arranged and neither tests a tool that makes booking easier

Online booking works in healthcare Zhao (2017) reviewed 21 web based medical appointment systems Online booking cut no shows with one system reporting 2% for web bookings against 8% for phone bookings and it also shortened waiting times raised satisfaction and reduced staff workload The review warns that providers worry about losing control of their schedules a concern lecturers are likely to share Guy (2012) add that automated reminders increase attendance These studies come from healthcare where appointments are more formal than a quick consultation so the idea still needs testing in education

People use systems that are useful and easy The Technology Acceptance Model suggests that people adopt a system when they find it useful and easy to use (Davis, 1989) Nielsen (1994) usability heuristics turn this into practical design rules such as always showing a clear confirmation and stopping a student from booking a slot that is already taken

Existing or Similar Systems

Microsoft Bookings offers a booking page that syncs with Outlook sends email reminders and creates Teams meetings It comes with Microsoft 365 business and education plans but SMS reminders only work in North America and the UK and need an extra Teams Premium licence Microsoft It is built for businesses serving customers so it has no idea of students lecturers or modules and setting it up needs administrator access

Calendly is a popular polished scheduling tool Its free plan allows only one event type and one calendar and reminders need the Standard plan at 10 US dollars per lecturer per month Calendly Students would need a separate link for every lecturer and booking data would sit with a third party

Moodle Scheduler is a Moodle plugin that lets teachers create slots for students to book with reminders booking limits and attendance tracking (Moodle, 2017) It suits teaching well but it only works inside one course needs an administrator to install it and is only an option for institutions that run Moodle

	Microsoft Bookings	Calendly	Moodle Scheduler	CampusBook
Lecturer sets slots	Yes	Yes	Yes	Yes
Student books and cancels	Yes	Yes	Yes	Yes
Free reminders	Email only	No paid plan	Yes	Yes email
Student and lecturer roles	No	No	Course roles	Yes
All lecturers in one place	Yes	No	No per course	Yes
Extra cost	Microsoft 365 licence	Per lecturer	Needs Moodle	None
Set up effort	Admin sets up a booking page	Each lecturer signs up	Admin installs it then added per course	Set up once for the campus
Where data is kept	Microsoft 365 tenant	Calendly servers	Moodle server	CTU server
Works on a phone	Yes in the browser	Yes browser and app	Through Moodle	Yes built for phones first
Main drawback	Business focus	Cost and data privacy	Tied to one course	Must be maintained

Lessons for CampusBook All three systems handle the basic act of booking well so CampusBook copies what works: clear open slots instant confirmation and reminders Where they fall short is fit Microsoft Bookings and Calendly are designed for businesses and Calendly charges per lecturer for reminders Moodle Scheduler fits teaching but stays inside a single course CTU current mix of email Teams WhatsApp and walk ins costs nothing but offers no shared view of availability and no record of meetings

Technology Review

The chosen tools are among the most widely used in the 2025 Stack Overflow Developer Survey so documentation and help are easy to find (Stack Overflow, 2025)

CampusBook stores names email addresses and booking details which counts as personal information under the Protection of Personal Information Act (Republic of South Africa, 2013) The system will collect only what it needs hash all passwords show each user only their own bookings and use made up data during development

Part	Choice	Why	Alternative
Front end	React	Builds pages from reusable components Meta used by about 45% of developers	Plain HTML and JavaScript harder to keep consistent
Back end	Node.js and Express	One language JavaScript across the whole team used by about 49% of developers	ASP.NET which adds a second language
Database	MySQL	Relational tables for users slots and bookings and the team already knows it	PostgreSQL more popular at about 56% but less familiar to us
Reminders	Email	Free and reaches everyone	SMS which costs per message
Method	Scrum	Sprints of a month or less with a review after each (Schwaber and Sutherland, 2020) which fits our weekly deadlines	Waterfall too rigid for weekly feedback

Research Gap

Research shows that consultations help students but are under used (Griffin, 2014) and that online booking with reminders reduces missed appointments in healthcare (Guy, 2012; Zhao, 2017) Existing tools serve businesses or single Moodle courses well but none gives CTU students one place to book any lecturer free of licence costs

Conceptual and Technical Direction

Two ideas guide the design From the Technology Acceptance Model every screen must make booking quicker than sending a message or students and lecturers will not switch (Davis, 1989) From the healthcare studies confirmations and reminders are what reduce missed meetings so they are core features rather than extras (Guy, 2012; Zhao, 2017) Lecturers keep full control of their own slots which answers the loss of control concern raised by Zhao (2017)

Technically CampusBook follows a simple three-tier design The React front end runs in the browser the Express API handles logins slots and bookings and the MySQL database stores the data with rules that stop a slot being booked twice A scheduled task sends reminder emails before each consultation

Chapter 3: Requirements and Methodology

Stakeholders and Users

Stakeholder	What they need from CampusBook	Access
Students	See open consultation times book change or cancel a slot and get a reminder before the meeting	Student account
Lecturers	Publish their available times and see who is coming and why	Lecturer account
CTU Training Solutions	Better planned consultations and more support for students before assessments	No admin dashboard in this version
Project supervisor	Review the requirements ethics and each weekly submission	Review only
Development team	Design build and test the system using made-up data	Development environment

Requirements Gathering

The first requirements came from four sources Our own experience of booking consultations at CTU feedback from our lecturer on the proposal the research comparison of Microsoft Bookings Calendly and Moodle Scheduler The features the three systems share became core requirements and the gaps they leave such as one place for every lecturer and free reminders shaped what makes CampusBook different

An anonymous survey of at least 10 CTU students will check how students book consultations now and what problems they run into the results will confirm or adjust these requirements

Functional Requirements

ID	Requirement	Priority
FR-01	Students and lecturers log in with their email and password	Must
FR-02	The system checks the user role so students and lecturers only reach their own pages and actions	Must
FR-03	Lecturers publish edit and remove their own future consultation slots	Must
FR-04	Students view the open slots of a chosen lecturer	Must
FR-05	Students book an open slot and give a short reason for the visit	Must
FR-06	Students change or cancel their own upcoming bookings	Must
FR-07	The system stops two students booking the same slot	Must
FR-08	The system sends an email confirmation after booking and a reminder before the meeting	Must
FR-09	Students see their own booking history and lecturers see the bookings for their slots	Must

Non-functional Requirements

ID	Requirement	Priority
NFR-01	A first-time student completes a booking in under 2 minutes	Must
NFR-02	The main pages load within 3 seconds in the team test set up	Must
NFR-03	Every page works on a phone at 375 pixels wide and on a laptop at 1366 pixels wide	Must
NFR-04	Passwords are stored as bcrypt hashes and never as plain text	Must
NFR-05	Each user can only see their own bookings or the bookings for their own slots	Must
NFR-06	Only the data needed for logging in and booking is stored with made-up data during development	Must
NFR-07	Errors explain what to fix and the form keeps what the user already typed	Should

User Stories

ID	User story	Requirements
US-01	As a student I want to see open slots so I can get help without messaging each lecturer	FR-04
US-02	As a student I want to book a slot with a reason so the lecturer knows what I need	FR-05 and FR-07
US-03	As a student I want to change or cancel my booking so my plans stay up to date	FR-06
US-04	As a lecturer I want to publish and manage my slots so I control my own time	FR-03
US-05	As a lecturer I want to see who booked and why so I can prepare for the meeting	FR-09
US-06	As a student or lecturer I want a confirmation and a reminder so no meeting is forgotten	FR-08

The main use case is booking a slot a student logs in chooses a lecturer and an open slot adds a reason and confirms the system checks that the slot is still open saves the booking and sends the confirmation If another student took the slot first the student is asked to choose another time

Development Methodology

The team uses Scrum with one week sprints where each sprint ends with a review so the next one can improve on it (Schwaber and Sutherland, 2020)

Sprint	Focus	Ends with
Sprint 0 16 to 19 Oct	Requirements SRS system design wireframes and repository set up	Week 3 submission
Sprint 1 20 to 23 Oct	Build the six must-have features and run the first tests	Week 4 submission
Sprint 2 24 to 29 Oct	User testing bug fixes final dissertation and presentation	Final submission and live demo

	Scrum	Waterfall
Feedback	Used every week	Mostly at the end
Changing requirements	Added to the next sprint	Hard once a phase is done
Risk	Problems show up early	Problems show up late in testing
Fit for this project	Matches the weekly deadlines	Too rigid for a two-week build

Ethics, Privacy and Security

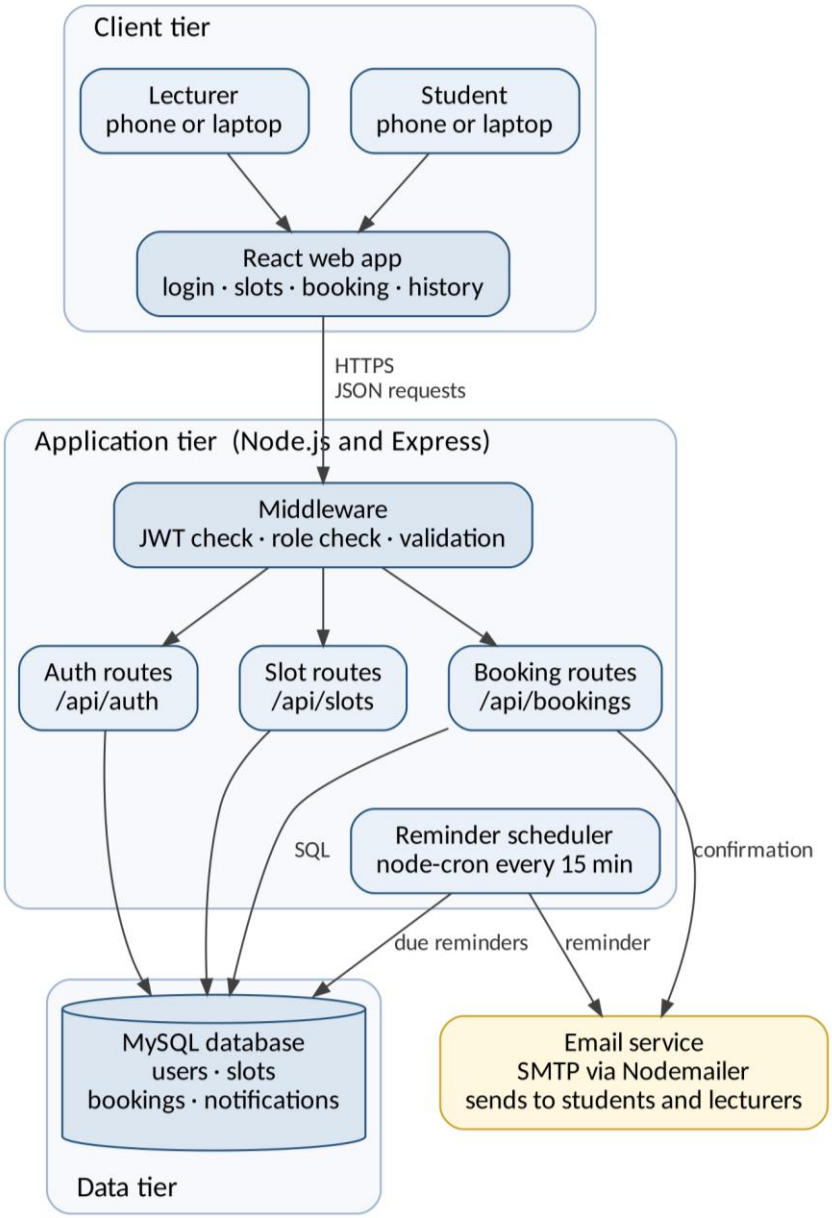
- Only made-up users and bookings are used during development never real CTU records
- The survey and user testing are voluntary with consent first and anonymous results as set out in Appendix B
- Research files are kept in a private folder and deleted once the project is marked
- Only the data needed for logging in and booking is collected in line with POPIA (Republic of South Africa, 2013)
- Passwords are hashed every request is checked for a valid token and role and all input is validated on the server
- A unique key in the database stops a slot ever having two confirmed bookings

Chapter 4: System Design

System Architecture

CampusBook uses a three tier architecture The client tier is a React web app that runs in the browser on a phone or laptop The application tier is a Node.js and Express API that holds all business rules The data tier is a MySQL database Splitting the system this way keeps the interface the rules and the data separate so each team member can work on one tier without breaking the others

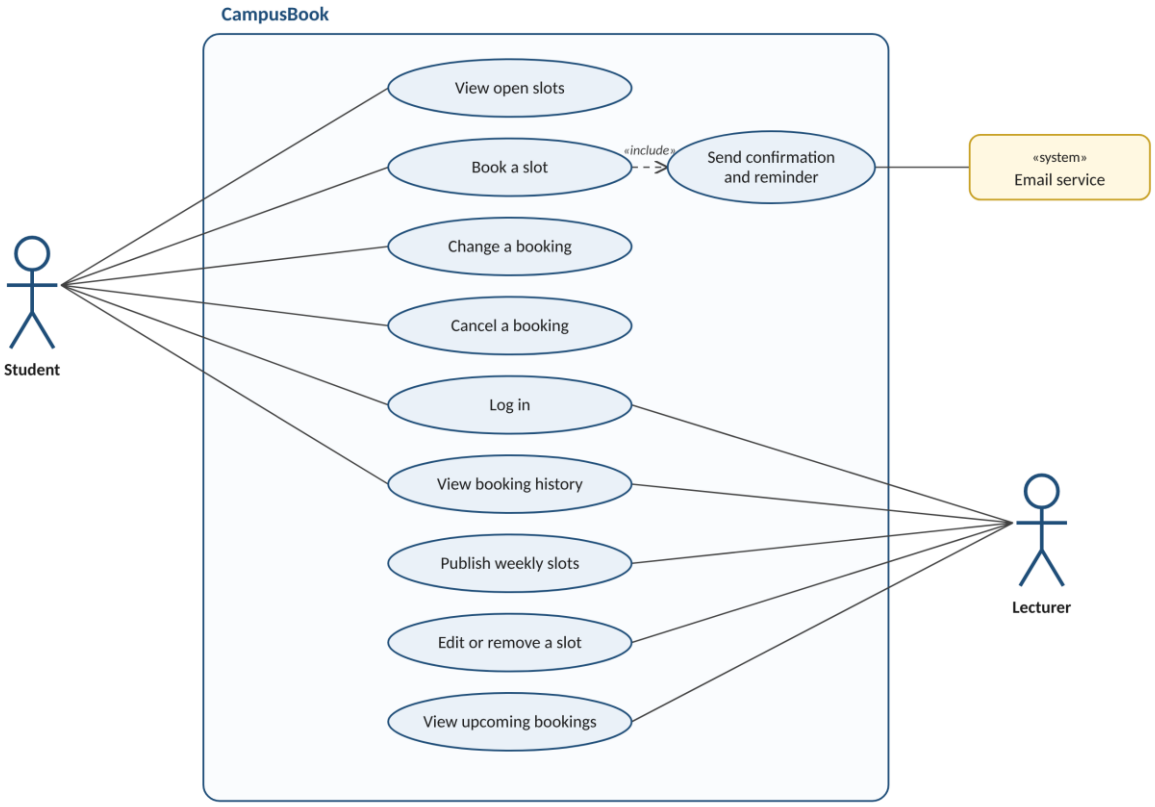
Every request from the browser goes to the API over HTTPS as JSON Middleware checks the login token and the user role and validates the input before any route runs the auth slot and booking routes then read and write the database A scheduler runs every 15 minutes finds bookings that are due a reminder and sends them through the email service Booking confirmations are sent straight after a booking is saved



UML Design

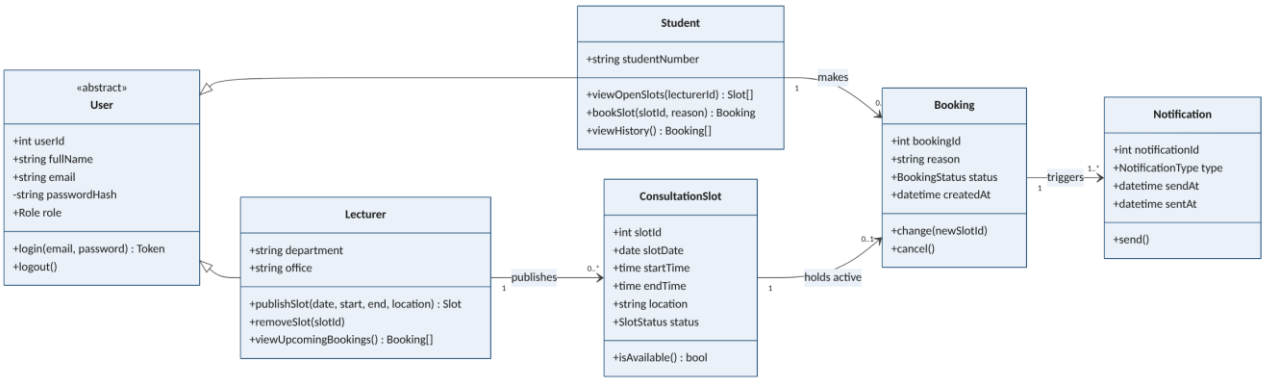
Use case diagram

The use case diagram shows what each user can do Students view open slots book change and cancel bookings Lecturers publish edit and remove slots and see upcoming bookings Both log in and view their booking history Booking a slot always includes sending a confirmation and reminder through the email service



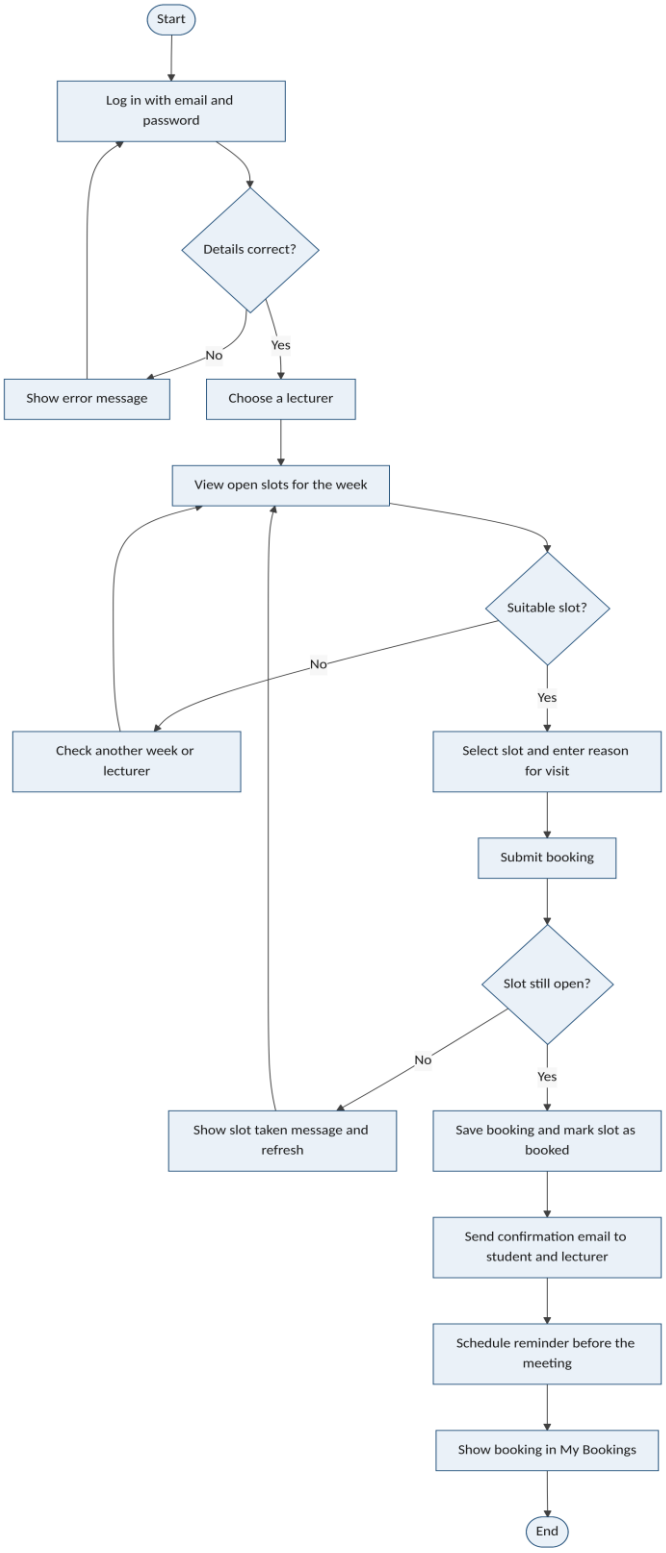
Class diagram

The class diagram shows the main objects in the system Student and Lecturer both inherit from User which holds the login details A lecturer publishes many consultation slots and a slot holds at most one active booking Each booking triggers one or more notifications such as the confirmation and the reminder



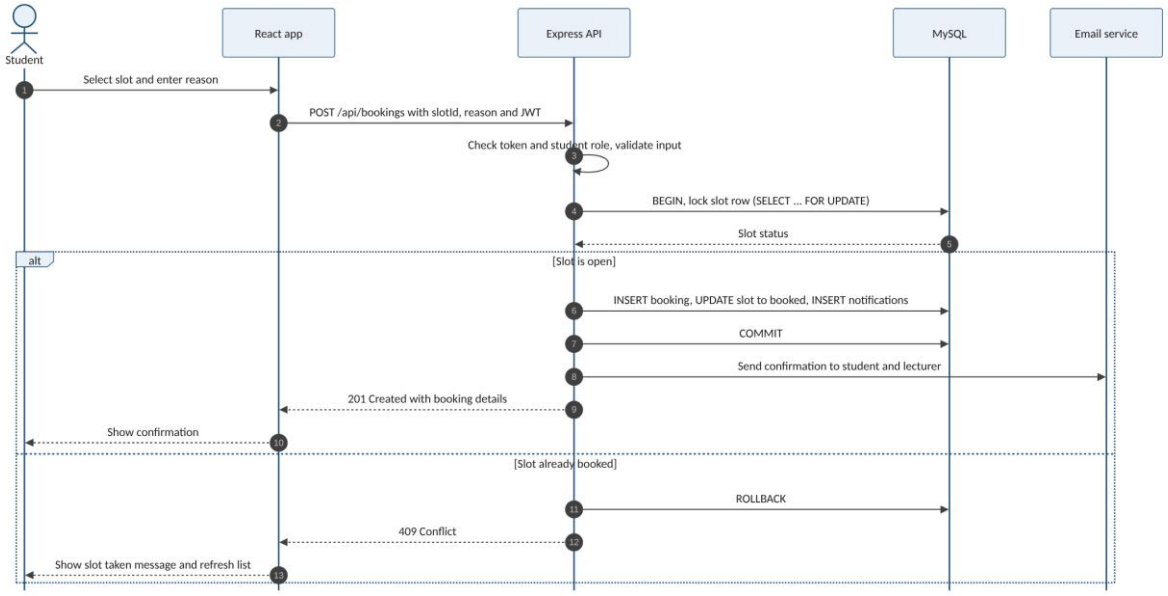
Activity diagram

The activity diagram follows a student booking a consultation It shows the two key decisions If no suitable slot is open the student can check another week or lecturer If another student takes the slot first the system shows a message and refreshes the list instead of creating a double booking



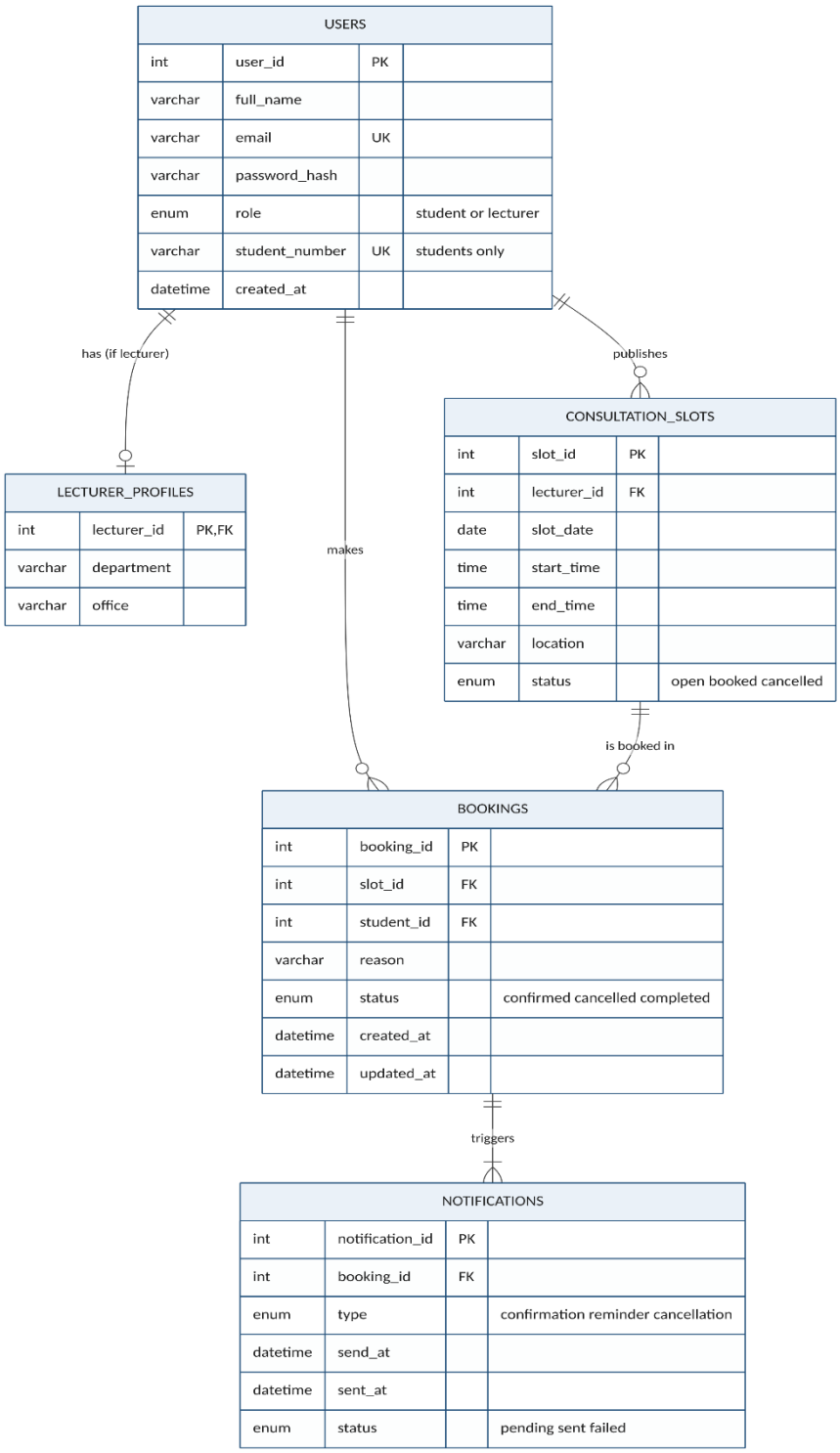
Sequence diagram

The sequence diagram shows the messages behind a booking The API locks the slot row inside a database transaction so two students pressing book at the same moment cannot both succeed If the slot is open the booking is saved the confirmation email is sent and the student sees a confirmation If not the transaction is rolled back and the student is told the slot was taken



Database Design

The database stores users lecturer details consultation slots bookings and notifications The ERD below shows the five tables their keys and how they relate



Relationships and cardinality

Relationship	Cardinality	Meaning
users to lecturer_profiles	One to zero or one	Only lecturers have a profile
users to consultation_slots	One to many	A lecturer publishes many slots
users to bookings	One to many	A student makes many bookings
consultation_slots to bookings	One to many	A slot can be booked again after a cancellation but only one booking is active at a time
bookings to notifications	One to one or more	Each booking has a confirmation and a reminder

Data dictionary

Column	Type	Null	Key	Description
users				
user_id	INT UNSIGNED	No	PK	Unique number for each user
full_name	VARCHAR(120)	No		Name shown in the app
email	VARCHAR(150)	No	Unique	Used to log in and receive emails
password_hash	VARCHAR(255)	No		bcrypt hash never the real password
role	ENUM	No		student or lecturer
student_number	VARCHAR(20)	Yes	Unique	Required for students only
lecturer_profiles				
lecturer_id	INT UNSIGNED	No	PK and FK	Links to users
department	VARCHAR(100)	No		Department the lecturer belongs to
office	VARCHAR(50)	Yes		Office or room
consultation_slots				
slot_id	INT UNSIGNED	No	PK	Unique number for each slot
lecturer_id	INT UNSIGNED	No	FK	Lecturer who owns the slot
slot_date	DATE	No		Day of the consultation
start_time and end_time	TIME	No		End must be after start
location	VARCHAR(100)	No		Room or Teams
status	ENUM	No		open booked or cancelled
bookings				
booking_id	INT UNSIGNED	No	PK	Unique number for each booking
slot_id	INT UNSIGNED	No	FK	Slot that was booked

Column	Type	Null	Key	Description
student_id	INT UNSIGNED	No	FK	Student who booked
reason	VARCHAR(255)	No		Reason for the visit
status	ENUM	No		confirmed cancelled or completed
active_slot_id	INT UNSIGNED	Yes	Unique	Filled only while confirmed stops double booking
notifications				
notification_id	INT UNSIGNED	No	PK	Unique number for each email
booking_id	INT UNSIGNED	No	FK	Booking the email belongs to
type	ENUM	No		confirmation reminder or cancellation
send_at and sent_at	DATETIME	sent_at only		When it is due and when it was sent
status	ENUM	No		pending sent or failed

Normalisation

The design is in third normal form Every table has a single column primary key and every other column depends only on that key Lecturer details sit in their own table so they are not repeated for students Lecturer and student names are stored once in users and joined when needed so changing a name never leaves old copies behind Notifications are kept apart from bookings so that one booking can have several emails without repeating booking data

Security and integrity

- Foreign keys stop a booking pointing at a slot or student that does not exist
- A unique key on active_slot_id means a slot can only ever have one confirmed booking
- Check constraints make sure end times are after start times and that students have a student number
- Passwords are stored as bcrypt hashes and each user can only reach their own bookings through the API
- Only made-up data is used during development in line with POPIA

SQL scripts

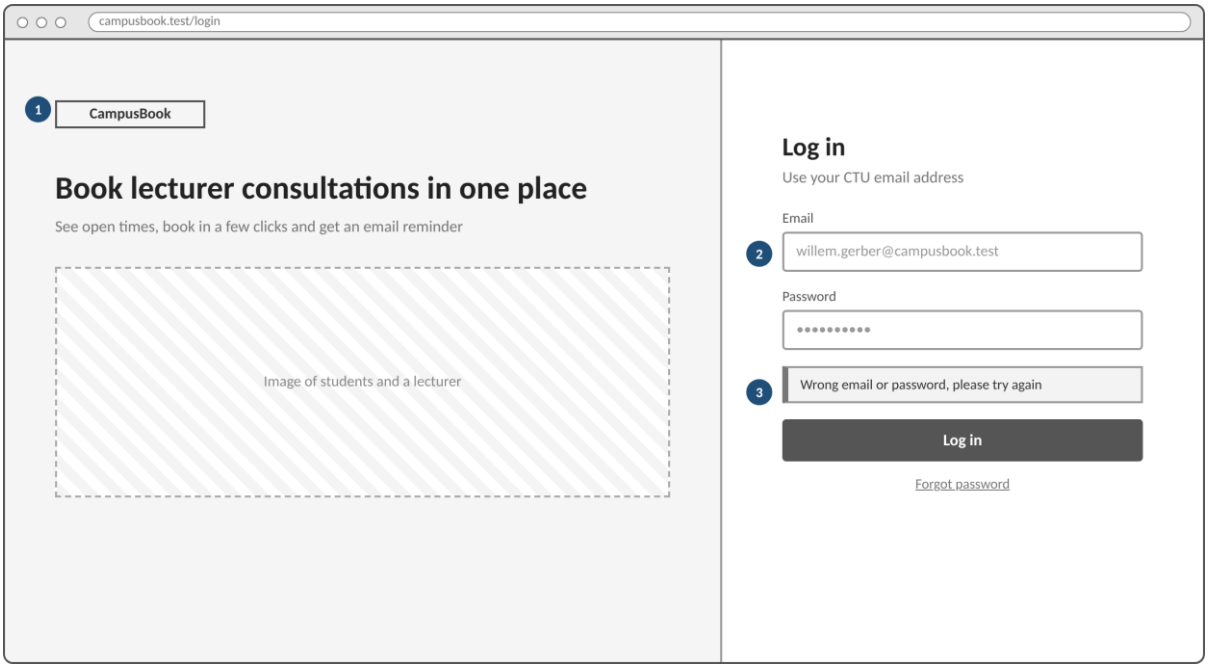
The full scripts are in the repository at database/schema.sql and database/seed.sql The part that prevents double booking is shown below

```
CREATE TABLE bookings (
    booking_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
    slot_id         INT UNSIGNED NOT NULL,
    student_id      INT UNSIGNED NOT NULL,
    reason          VARCHAR(255) NOT NULL,
    status          ENUM('confirmed','cancelled','completed') DEFAULT 'confirmed',
    active_slot_id  INT UNSIGNED AS (IF(status = 'confirmed', slot_id, NULL)) STORED,
    PRIMARY KEY (booking_id),
    UNIQUE KEY uq_one_active_booking_per_slot (active_slot_id),
    CONSTRAINT fk_booking_slot FOREIGN KEY (slot_id) REFERENCES consultation_slots (slot_id),
    CONSTRAINT fk_booking_student FOREIGN KEY (student_id) REFERENCES users (user_id)
);
```

User Interface Design

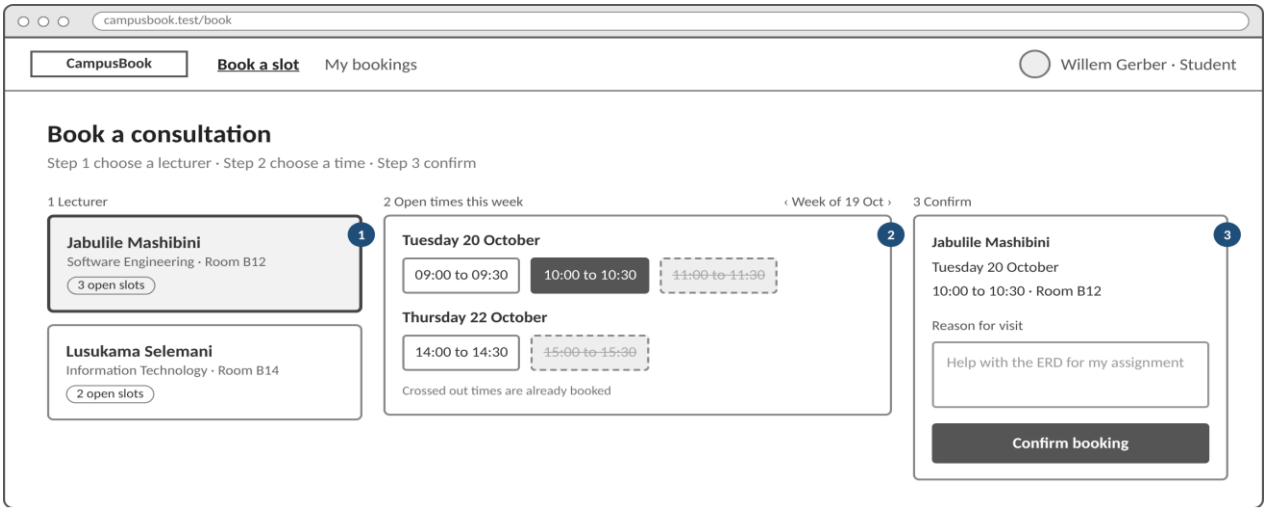
The screens below are low fidelity wireframes that show the layout and flow of each page before it is built in React in Week 4 The numbered markers point to the design choices explained under each screen The layout follows the usability heuristics from Chapter 2 (Nielsen, 1994) and keeps every task to as few steps as possible so that booking is quicker than sending a message (Davis, 1989)

Login



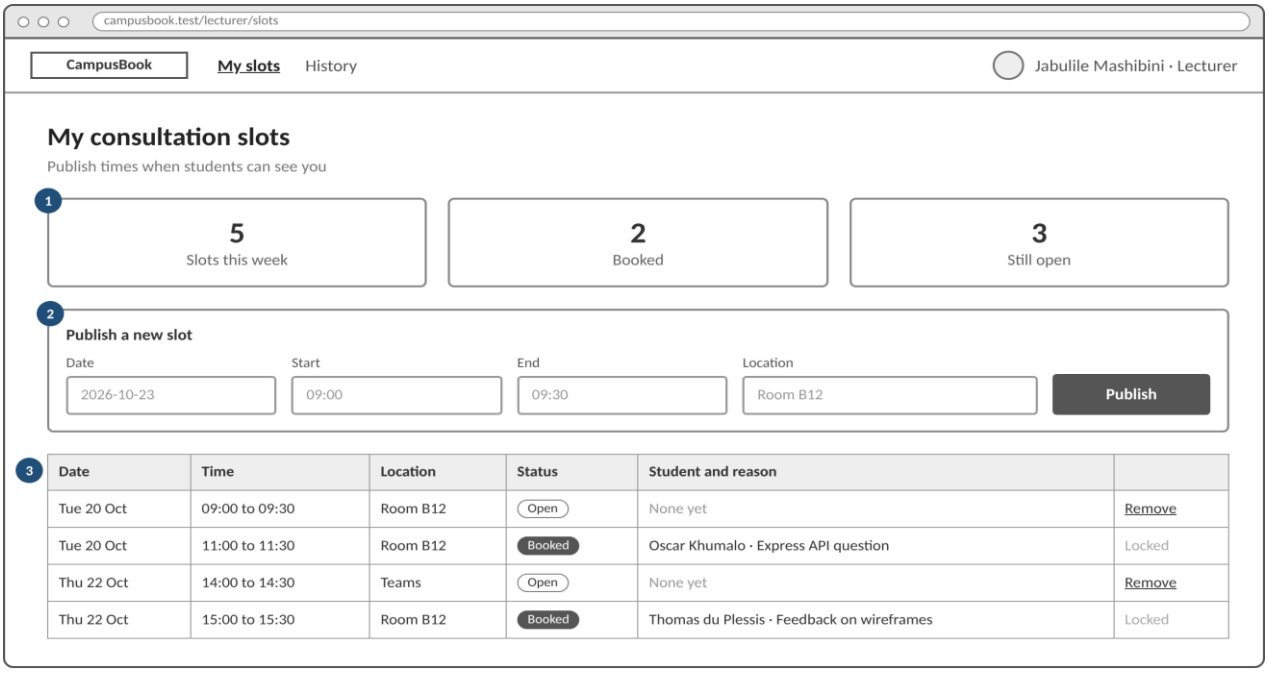
1. The left side explains in one line what CampusBook does so new users know they are in the right place
2. Labels sit above each field and the email field hints that a CTU address is needed
3. Errors appear in plain words right above the button so users can recognise and recover from a mistake

Student booking screen



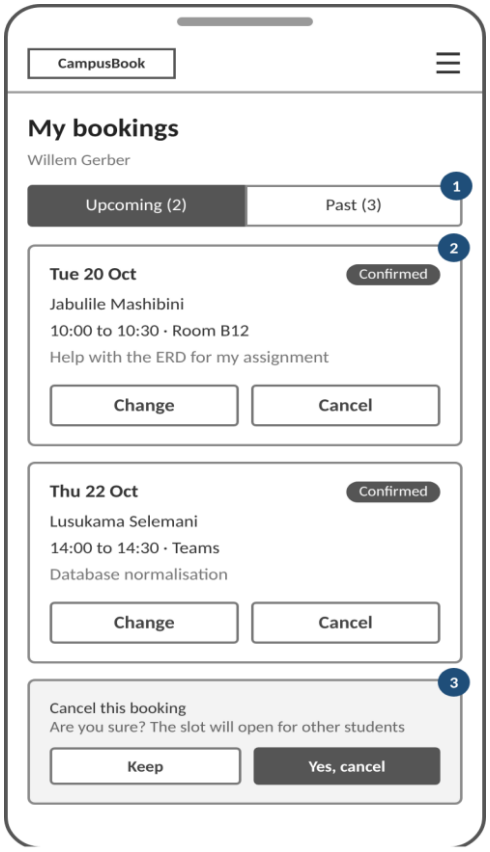
1. Lecturer cards show the department room and number of open slots so students can choose quickly
2. Open times are grouped by day and times that are already taken are crossed out instead of hidden which prevents errors
3. A summary repeats the lecturer date time and place before the student confirms and the reason for the visit is required

Lecturer slots screen



1. Three counters show the week at a glance so lecturers always know the status of the system
2. A new slot is published with four fields on one line which keeps the most common lecturer task short
3. The table lists every slot with the student and reason and booked slots are locked so a confirmed meeting cannot be removed by accident

My bookings on a phone



1. Upcoming and past bookings sit in two tabs so the list stays short on a small screen
2. Each card shows the date lecturer time place and reason with large buttons that are easy to tap
3. Cancelling asks for confirmation first and explains that the slot will open for other students

Technology Stack

Part	Choice	Why it was chosen
Front end	React with Vite	Reusable components keep the four screens consistent and Vite gives fast reloads while developing
Routing	React Router	Separate pages for students and lecturers with protected routes
Back end	Node.js with Express	The whole team works in one language JavaScript and Express keeps the API small and readable
Database	MySQL 8	Relational tables foreign keys and transactions suit bookings and the team already knows SQL
Security	bcryptjs and JSON Web Tokens	Passwords are hashed and every request carries a signed token with the user role
Email	Nodemailer and node-cron	Free email over SMTP with a scheduled job for reminders
Version control	Git and GitHub	Feature branches and pull requests show each member contribution

Development Evidence

```
CampusBook/
├── backend/             Express REST API
│   └── src/
│       ├── routes/      auth.js, slots.js, bookings.js
│       ├── middleware/  auth.js, validate.js
│       ├── services/    mailer.js
│       ├── jobs/        reminders.js
│       ├── db.js
│       └── server.js
├── frontend/            React web app
│   └── src/pages/       Login, BookSlot, MyBookings, LecturerSlots
├── database/            schema.sql, seed.sql
├── docs/                diagrams and wireframes
└── README.md
```

Item	Status	Owner
Project structure and README	Done	Willem Gerber
Database schema script	Done and tested	Gerhardus van den Berg
Test data script	Done	Gerhardus van den Berg
Login with student and lecturer roles	Planned for Week 4	Oscar Khumalo
Slots and booking endpoints	Planned for Week 4	Oscar Khumalo and Willem Gerber
React pages and mobile layout	Planned for Week 4	Thomas du Plessis
Email confirmation and reminders	Planned for Week 4	Gerhardus van den Berg

Reference List

Calendly (n.d.) *Pricing*.
https://calendly.com/pricing

Davis, F.D. (1989) 'Perceived usefulness, perceived ease of use, and user acceptance of information technology', *MIS Quarterly*, 13(3), pp. 319-340.
doi:10.2307/249008

Griffin, W., Cohen, S.D., Berndtson, R., Burson, K.M., Camper, K.M., Chen, Y. and Smith, M.A. (2014) 'Starting the conversation: an exploratory study of factors that influence student office hour use', *College Teaching*, 62(3), pp. 94-99.
doi:10.1080/87567555.2014.896777

Guy, R., Hocking, J., Wand, H., Stott, S., Ali, H. and Kaldor, J. (2012) 'How effective are short message service reminders at increasing clinic attendance? A meta-analysis and systematic review', *Health Services Research*, 47(2), pp. 614-632.
https://pubmed.ncbi.nlm.nih.gov/22091980/

Meta (n.d.) *Quick start: React*.
https://react.dev/learn

Microsoft (n.d.) *Microsoft Bookings service description*.
https://learn.microsoft.com/en-us/office365/servicedescriptions/microsoft-bookings-service-description

Moodle (2017) *Scheduler module*. MoodleDocs.
https://docs.moodle.org/405/en/Scheduler_module

Nielsen, J. (1994) *10 usability heuristics for user interface design*. Nielsen Norman Group.
https://www.nngroup.com/articles/ten-usability-heuristics/

Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.
https://www.gov.za/documents/protection-personal-information-act

Schwaber, K. and Sutherland, J. (2020) *The Scrum guide*.
https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

Stack Overflow (2025) *2025 Developer Survey: technology*.
https://survey.stackoverflow.co/2025/technology

Statistics South Africa (2024) *General Household Survey 2023*. Pretoria: Statistics South Africa.
https://www.statssa.gov.za/publications/P0318/

Weimer, M. (2015) 'Why students don't attend office hours', *The Teaching Professor*, 21 January.
https://www.teachingprofessor.com/articles/teaching-professor-blog/students-dont-attend-office-hours

Zhao, P., Yoo, I., Lavoie, J., Lavoie, B.J. and Simoes, E. (2017) 'Web-based medical appointment systems: a systematic review', *Journal of Medical Internet Research*, 19(4), e134.
doi:10.2196/jmir.6747

Appendix A: Project Charter and Work Plan

Project Overview

CampusBook is a web based consultation booking system for CTU Training Solutions Students currently arrange consultations through email Teams WhatsApp and walk-ins which leads to missed and unplanned meetings The project will build and evaluate a system that gives students and lecturers one place to manage consultations

Project	CampusBook
Client	CTU Training Solutions students and lecturers
Supervisor	Jabulile Mashibini
Group Lead	Willem Gerber
Method	Scrum with one-week sprints
Duration	16 October to 29 October 2026

Aim and Objectives

The aim is to design develop and evaluate CampusBook so that fewer consultations at CTU are missed or left unplanned the five objectives are to analyse current practice compare existing systems build the six must have features test the system and evaluate it with real users

Scope and Deliverables

The project covers one CTU campus student and lecturer roles and the six must have features It excludes SMS video calls native apps and links to CTU live systems The deliverables are:

- A working web application with source code on GitHub
- A dissertation covering Chapters 1 to 6 with testing evidence
- A live demonstration and presentation on 29 October 2026

Team and Roles

Member	Role	Responsible for
Willem Gerber	Group Lead and Scrum Master	Stand ups GitHub board booking logic compiling the dissertation
Oscar Khumalo	Back-end developer	Express API login and roles back-end write-up
Thomas du Plessis	Front-end developer	React pages mobile layout and wireframes
Lebohang Tlaitlai	Testing and research lead	Student survey literature review test cases
Gerhardus van den Berg	Database developer	MySQL schema and ERD email reminders

Milestones

Sprint	Main tasks	Milestone
Sprint 0 16 to 19 Oct	Chapters 1 to 3 requirements system design charter ethics form GitHub set-up and first commits	Week 2 and 3 PDFs due Monday 19 Oct 12pm
Sprint 1 20 to 23 Oct	Database login slots and booking reminders history mobile layout student survey testing	Working app and Chapters 4 and 5 due Friday 23 Oct 12pm
Sprint 2 24 to 29 Oct	User testing bug fixes full dissertation Turnitin check presentation practice	Final submission and live demo Thursday 29 Oct

Risks and Assumptions

Risk	Level	How we handle it
Not enough time to build every feature	High	Build the must-haves first and commit daily
Work falls on one or two people	High	Clear task owners checked at every stand up and on GitHub
Some members are new to React or Express	Medium	Pair programming and a shared starter project
Too few survey or test participants	Medium	Share the survey early in class and on Teams

Appendix B: Ethical Clearance Confirmation

Project Title	CampusBook: a lecturer consultation booking system for CTU Training Solutions
Group Members	Willem Gerber Oscar Khumalo Thomas du Plessis Lebohang Tlaitlai Gerhardus van den Berg
Module	Software Engineering 2
Supervisor	Jabulile Mashibini
Week / Stage	Week 3
Submission Date	Monday 19 October 2026

Data and Participants

Activity	Involved	Details
Personal data	Yes	Names emails and bookings with only made-up data during development
Interviews	No	No formal interviews our lecturer reviewed the proposal and supported the idea
Survey	Yes	Anonymous at least 20 CTU students no names or student numbers
User testing	Yes	At least 5 students and 2 lecturers with a short questionnaire
Minors or sensitive topics	No	Adults only questions are only about booking consultations

Ethical Considerations

- Consent: every participant gets a short information sheet and takes part voluntarily and can stop at any time User testing participants sign a consent form and survey participants tick a consent box before starting
- Confidentiality: survey answers are anonymous testing notes use codes such as S1 instead of names and results are only reported as totals or anonymous quotes
- Secure handling: research data is kept in a private password-protected folder and deleted once the project is marked
- Data minimisation: the system only collects what it needs hashes passwords and shows users only their own bookings in line with POPIA

Appendix C: Literature Review Matrix

Source	Focus and method	Key finding	Use and limitation
Griffin (2014)	Office hour use student survey	Most barriers are outside the lecturer control	Shows the problem but only one US university
Weimer (2015)	Office hour attendance survey summary	66% of students never attended	Shows the scale but not peer reviewed
Zhao (2017)	Online medical booking systematic review	Fewer no-shows shorter waits less staff work	Supports online booking but healthcare only
Guy (2012)	SMS reminders meta-analysis	Reminders increase attendance	Supports reminders but tested SMS not email
Davis (1989)	Technology acceptance model	Usefulness and ease of use drive adoption	Guides evaluation but an older workplace study
Nielsen (1994)	Usability heuristics	Ten rules for usable interfaces	Guides design but based on expert opinion
Microsoft (n.d.)	Microsoft Bookings product documentation	Strong Outlook and Teams links business focus	Comparison system vendor source
Calendly (n.d.)	Calendly pricing page	Free plan limited reminders paid	Comparison system vendor source
Moodle (2017)	Moodle Scheduler documentation	Built for teaching but tied to one course	Comparison system page may be dated
Stack Overflow (2025)	Developer survey	React Node.js and MySQL are widely used	Supports the stack but a self-selected sample

In line with industry requirements, CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (only selected registered campuses) (No 2014/HE07/004).
Directors: J Newby, E Ferreira, B Brown. | Co. Reg No: 2005/038944/07
ctutraining.ac.za | 0861 100 395 | enquiry@ctutraining.co.za

Appendix D: Software Requirements Specification

Project	CampusBook
Client	CTU Training Solutions
Version	Draft 1.0
Date	Monday 19 October 2026

Purpose and Scope

This SRS sets out what CampusBook must do and how each requirement will be tested CampusBook is a responsive web app for one CTU campus where students book change and cancel consultations lecturers manage their slots and both see their booking history SMS video calls native apps admin dashboards and links to live CTU systems are out of scope

System Context and Constraints

- Built with React Node.js with Express and MySQL as set out in Chapter 4
- Two roles only student and lecturer
- Made-up data only during development and testing
- Email reminders need an SMTP service that is set up and tested before the demo
- The app must be working by 23 October 2026

Functional Requirements

ID and name	Requirement	Priority	Acceptance test
FR-01 Login	Students and lecturers log in with a valid email and password	Must	Valid details log in and wrong details show a general error that does not say which part was wrong (TC-01)
FR-02 Role checks	Student and lecturer actions are checked on the server	Must	A request for the wrong role is rejected even when sent straight to the API (TC-02)
FR-03 Manage slots	Lecturers publish edit and remove their own future slots that are not booked	Must	Changes show in the open slots and a lecturer cannot change another lecturer slot (TC-03)
FR-04 View slots	Students view the open future slots of a chosen lecturer	Must	Only future slots that are not booked are listed (TC-04)
FR-05 Book a slot	Students book an open slot with a short reason	Must	The booking is saved with the student slot and reason and a confirmation is shown (TC-05)
FR-06 Change or cancel	Students change or cancel their own upcoming bookings	Must	The old slot opens again after a change or cancellation (TC-06)
FR-07 No double booking	Two confirmed bookings for one slot are never possible even at the same moment	Must	Two requests at the same time give only one confirmed booking (TC-07)

ID and name	Requirement	Priority	Acceptance test
FR-08 Emails	A confirmation is sent after booking and a reminder before the meeting	Must	Emails show the right details and failed emails are logged (TC-08)
FR-09 History	Students see their own bookings and lecturers see the bookings for their slots	Must	Upcoming and past bookings show with no data from other users (TC-09)

Non-functional Requirements

ID and name	Requirement	How it is checked	Test
NFR-01 Usability	A first-time student books in under 2 minutes	Timed task with test participants	TC-10
NFR-02 Speed	Main pages load in under 3 seconds	Browser timings on the team test set up	TC-11
NFR-03 Phone and laptop	Pages work at 375 and 1366 pixels wide with no sideways scrolling	Use both screen sizes	TC-12
NFR-04 Passwords	Passwords are stored as bcrypt hashes	Check the database and login code	TC-13
NFR-05 Access control	Users only see their own records or bookings for their own slots	Try to reach another account through the API	TC-14
NFR-06 Data minimisation	Only data needed for login and booking is stored with made-up data	Review the schema and test data	TC-15
NFR-07 Error handling	Errors explain what to fix and keep what was typed	Submit empty and wrong input	TC-16

Data Requirements

CampusBook stores five tables users lecturer_profiles consultation_slots bookings and notifications A lecturer has many slots a student has many bookings a slot has only one active booking and each booking has one or more emails The full ERD and data dictionary are in Chapter 4

Acceptance Targets

- Every must-have test case passes
- A booking takes under 2 minutes
- Pages load within 3 seconds
- At least 80% of 5 students and 2 lecturers rate the system easy to use

Appendix E: Weekly Progress Report and Registers

Weekly Progress Report

Reporting period	16 to 19 October 2026 Sprint 0
Overall status	On track for the Week 3 submission
Done this week	Chapters 1 and 2 updated Chapter 3 and the SRS Chapter 4 design with all diagrams and wireframes repository structure schema and test data scripts
Still open	First commits by every member and the student survey
Next milestone	Working app Chapters 4 and 5 and the video by Friday 23 October 12pm

Team Progress

Member	This week	Next week
Willem Gerber	Repository structure architecture diagram technology stack compiling Week 2 and Week 3	Booking logic merging pull requests walkthrough video
Oscar Khumalo	Functional and non-functional requirements sequence diagram	Login and role endpoints slot and booking API
Thomas du Plessis	Wireframes use case and activity diagrams	React pages and mobile layout
Lebohang Tlaitlai	Chapter 3 SRS and the student survey	Test plan test cases and survey results
Gerhardus van den Berg	ERD class diagram schema and test data scripts	Database connection and email reminders

Risk Register

ID	Risk	Likelihood and impact	How we handle it	Owner
R-01	Not enough time to build all six features	High and High	Build the must-haves first and check commits daily	Willem
R-02	Work falls on one or two members	Medium and High	Clear owners for every task tracked on GitHub	Willem
R-03	Some members are new to React or Express	Medium and Medium	Pair programming and the shared project structure	Oscar and Thomas
R-04	Too few survey or test participants	Medium and Medium	Share the survey early in class and on Teams	Lebohang
R-05	Email reminders not working in time	Medium and High	Set up the email service early and test it	Gerhardus
R-06	Double booking or one user seeing another user data	Medium and High	Database unique key role checks and access tests	Oscar

Issue Register

ID	Issue	Status	Action
I-01	Survey target was 10 students	Closed	Both now say at least 10 students
I-02	Supervisor approval for the survey and user testing	Open	Confirm with Jabulile Mashibini before sending the survey
I-03	First commits from every member	Open	Each member pushes their first commit before Monday 19 October 12pm

Change Register

ID	Change	Reason	Status
C-01	Lecturer interviews replaced by lecturer feedback on the proposal	No formal interviews were held	Done in Week 2
C-02	Survey target set to at least 10 students everywhere	Two different numbers were used	Done in Week 3
C-03	App screenshots replaced by wireframes	The app is built in Week 4	Done in Week 3
C-04	Test set up for the page load target written down	Makes NFR-02 repeatable	Planned for Week 4

Updated Work Plan

Dates	Work	Who	Milestone
16 to 19 Oct	Requirements SRS design wireframes charter ethics and repository set up	All led by Willem	Week 3 due 19 Oct 12pm
20 to 21 Oct	Database and API React pages and the student survey	Oscar ,Thomas and Lebohang	First routes and pages
22 to 23 Oct	Booking reminders history and first tests	Development and testing team	Week 4 due 23 Oct 12pm
24 to 27 Oct	User testing bug fixes security and usability checks	All	Test evidence
28 to 29 Oct	Final dissertation evidence and presentation	All	Final submission and demo 29 Oct

In line with industry requirements, CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (only selected registered campuses) (No 2014/HE07/004).
Directors: J Newby, E Ferreira, B Brown. | Co. Reg No: 2005/038944/07
ctutraining.ac.za | 0861 100 395 | enquiry@ctutraining.co.za